OPERATING SYSTEMS (MOS)

Complete Handwritten Lecture Notes Transcribed to Digital Format

High-Contrast Dark Edition • KTU B.Tech S5 Computer Science & Engineering

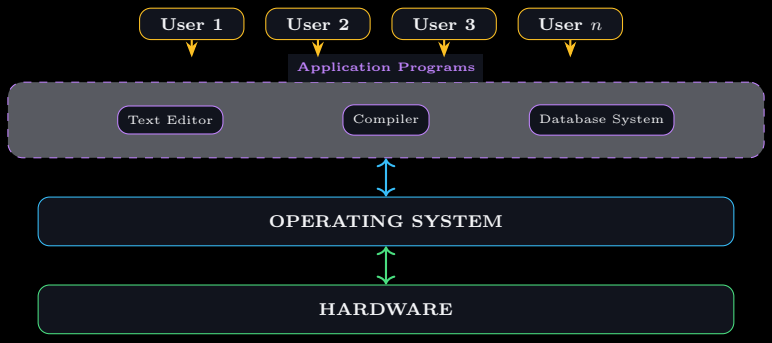
1 Introduction to Operating Systems

Operating System Definition: An **Operating System (OS)** is a software that manages and handles hardware and software resources of a computing device. It acts as an interface between the user and hardware.

Key Roles & Functions

- **Resource Manager:** Manages computer resources such as CPU, memory (RAM), and files/storage.
- **User-Hardware Interface:** Acts as an interface between user applications and the raw hardware.
- **Performs Core Functions:** Executes core functions like process, memory, and file management.
- **Organizes Resources:** Organizes system resources similar to different departments in a government.
- **Real-World Examples:** Linux, Unix, Windows 11, MS-DOS, Android, macOS, iOS.

Abstract View of System Components



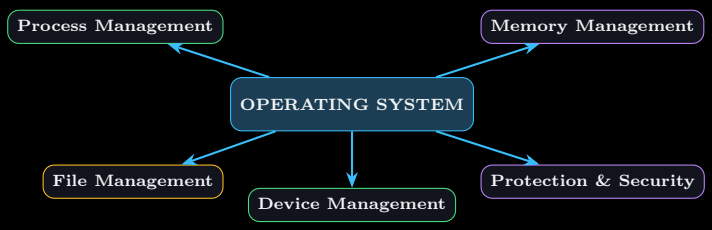
1.1 Components of an OS

There are two basic components of an OS:

1. **Shell:** It is the outermost layer of the operating system and handles user interaction. It interprets input for the OS and handles the output from the OS.
2. **Kernel:** It is the core component of the OS. The kernel is the primary interface between the operating system and hardware.

2 Functions of an Operating System

The OS controls all computer resources, coordinates program execution, hides system complexity, ensures security, and improves resource utilization.



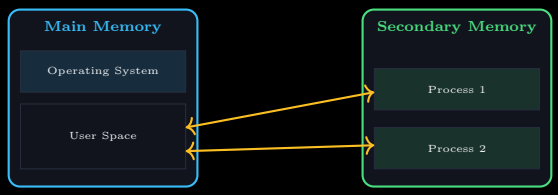
2.1 Process Management

Process management in an operating system is about managing processes. A **process** is a running program.

- **a) Scheduling:** Decides which process uses the CPU next.
- **b) Synchronization:** Ensures orderly execution using locks, semaphores, or monitors to prevent race conditions.
- **c) Deadlock Handling:** Detects & prevents situations where processes wait forever for resources.
- **d) Inter-Process Communication (IPC):** Allows processes to exchange data via shared memory or message passing.

2.2 Memory Management

Memory management in an operating system controls how data is stored and organized in main memory and secondary storage, ensuring programs get the memory they need and freeing it when no longer used.



Key Activities in Memory Management

- **Allocation & Deallocation:** Assign and free memory as needed.
- **Protection:** Prevents processes from interfering with each other's memory.
- **Virtual Memory:** Uses disk space as extra memory to run large programs.
- **Fragmentation Handling:** Reduces wasted space through compaction.
- **Disk Management:** Handles file storage, free space, disk scheduling, and backups.

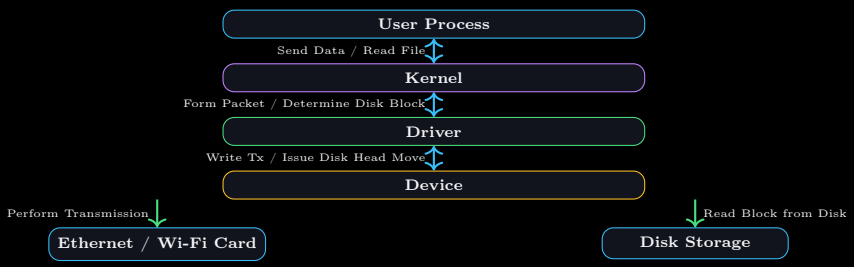
2.3 File System Management

File management in an OS organizes and controls how data is stored, named, and accessed on storage devices, making it easy for users to find and use their files.

- **File Attributes:** Names, type, size, and permissions.
- **File Types:** Text, binary, and executable files.
- **Operations:** Create, read, write, and delete files.
- **Access Methods:** Sequential, direct, and indexed access for faster retrieval.

2.4 Device Management (I/O System)

Device management in an OS controls how the computer communicates with hardware devices like printers or disks, using drivers and techniques to ensure smooth and efficient operation.



Major Components in Device Management

- **Device Drivers:** Interface between hardware and OS.
- **Buffering & Caching:** Temporarily store data to match device speeds and improve performance.
- **Spooling:** Spooling manages data waiting to be processed, particularly in devices like printers. The OS places print jobs in a **spool** (a temporary storage area), allowing the CPU to continue other tasks while the printer works through the queue.

2.5 Protection and Security

Protection and security mechanisms in an OS are designed to safeguard system resources from unauthorized access or misuse.

- These mechanisms control which processes or users can access specific resources (such as memory, files, and CPU time).
- Ensures that only authorized users can perform specific actions.
- **Access Control:** Limits user and process permissions.
- **Authentication:** Verifies users through credentials (UID / SID).
- **Resource Protection:** Prevents misuse of files, memory, or devices.
- **Security:** Guards against malware, denial-of-service (DoS) attacks, and data theft.

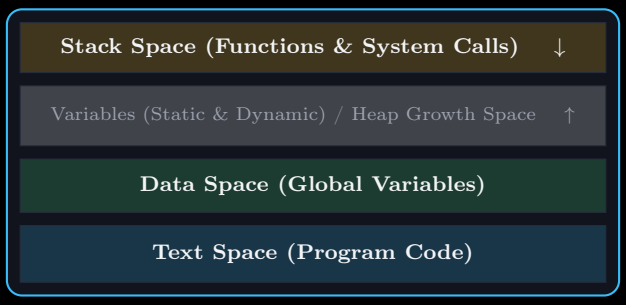
3 Processes and Process Lifecycle

3.1 Process Concept

- A **process** is a program in execution.
- A process is more than the program code, sometimes known as the **text section**.
- It includes the current activity as represented by the value of the **Program Counter (PC)** and processor registers.
- A process includes the **process stack** (temporary data: parameters, return addresses, local variables) and a **data section** (global variables).

Process Address Space Layout

Each process runs in its own virtual memory address space:

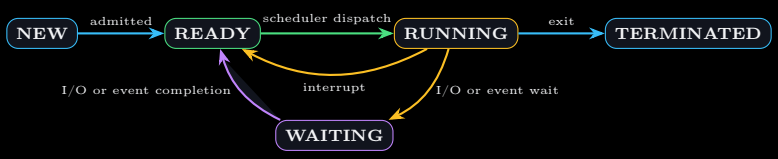


3.2 Process States

The state of a process is defined by its current activity. Each process may be in one of the following states:

- **New:** The process is being created.
- **Running:** Instructions are being executed.
- **Waiting:** The process is waiting for some event to occur (e.g., I/O completion or signal).
- **Ready:** The process is waiting to be assigned to a processor.
- **Terminated:** The process has finished execution.

5-State Process Transition Diagram



3.3 Process Control Block (PCB)

Each process is represented in the operating system by a **Process Control Block (PCB)**:

Process Control Block (PCB) Fields

- **Pointer:** Pointer to next PCB in scheduling queues.
- **Process State:** New, ready, running, waiting, terminated.
- **Process Number (PID):** Unique integer identifier.
- **Program Counter:** Indicates the address of the next instruction to be executed.
- **Registers:** Accumulator, index registers, stack pointers, general purpose registers, condition code info.
- **Memory Limits:** Values of base and limit registers, page tables, or segment tables.
- **List of Open Files:** Table of file handles open by this process.
- **CPU Scheduling Information:** Process priority, pointers to scheduling queues, and scheduling parameters.
- **Accounting Information:** CPU time used, time limits, execution/account numbers, clock time / timestamps.
- **I/O Status Information:** Tracks all input and output operations and devices linked to the process.

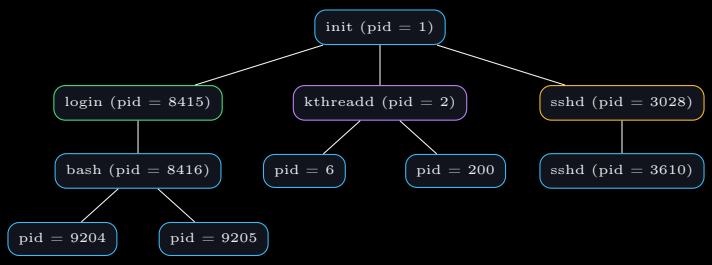
4 Process Operations & CPU Schedulers

4.1 Process Creation

During the process lifecycle, a process may create several new processes. The creating process is called a **parent process**, and the new processes are called **children** of that process. Each of these new processes may in turn create other processes, forming a **tree of processes**.

Process Tree Hierarchy (pstree)

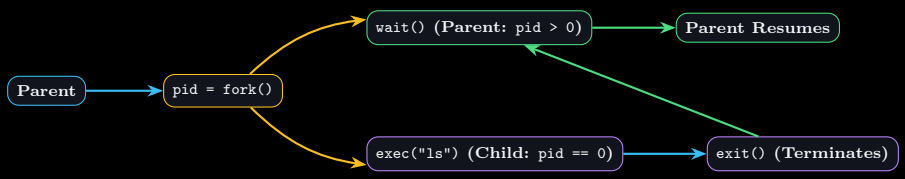
Operating systems identify processes with a unique **Process Identifier (PID)**, which is typically an integer number.



Execution & Address Space Alternatives

Execution Possibilities: (1) Parent continues to execute concurrently with its children, OR (2) Parent waits until some or all of its children have terminated.
Address Space Possibilities: (1) Child process is a duplicate of the parent process, OR (2) Child process has a new program loaded into it.
Resource Allocation: Child may obtain resources directly from the OS, or be constrained to a subset of the parent process's resources.

Process Creation using System Calls: `fork()`, `exec()`, `wait()`, `exit()`



4.2 Process Termination

A process terminates when it finishes executing and asks the operating system to delete it by using the `exit()` system call.

- Returns a status value to parent process (via `wait()` system call).
- All resources (memory, open files, I/O buffers) are deallocated by the OS.

A parent may terminate children if resource limits are exceeded or the assigned task is no longer required. If a parent terminates, all its children must also be terminated (**cascading termination**).

4.3 Schedulers

A **scheduler** manages multitasking by deciding which processes gain access to the CPU:

1. **Long-Term (Job) Scheduler:** Selects programs from secondary memory (disk) and loads them into RAM. Controls **degree of multiprogramming** and balances CPU-bound vs. I/O-bound processes.
2. **Short-Term (CPU) Scheduler:** Extremely fast and frequent; selects the next ready process in RAM to execute on the CPU.
3. **Medium-Term Scheduler:** Manages memory by temporarily **swapping** blocked or idle processes out to the disk (to free up RAM) and swapping them back in when they resume.

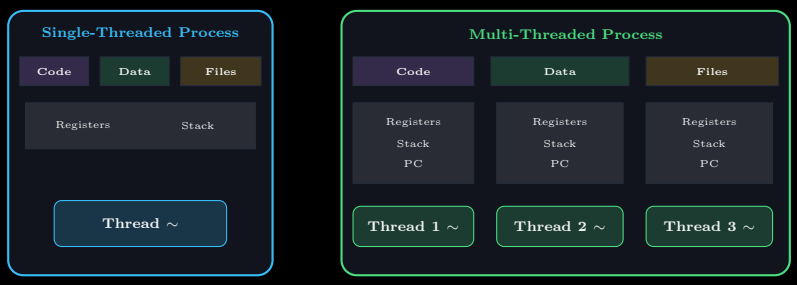
5 Threads and Multithreading

5.1 Thread Concept

A **thread** is a lightweight process (LWP) because it is smaller and faster.

- Allows multiple tasks to run simultaneously, improving program efficiency.
- **Single-core:** Illusion of parallelism (concurrency). **Multicore:** True parallel execution across cores.
- **Thread Private State:** Program Counter (PC), register set, and stack space.
- **Shared State:** Code, data, and OS resources (files) within the same process.

Single-Threaded vs. Multi-Threaded Process Architecture



5.2 Need of Threads

1. **Improves Performance:** Multiple tasks run simultaneously (parallel/interleaved), making programs faster.
2. **Increase Responsiveness:** If one thread is busy, another handles user input, keeping UI responsive.
3. **Enable Concurrency:** Multiple operations (saving, processing, input) occur simultaneously.
4. **Better CPU Utilization:** On multicore systems, threads run on different cores to maximize CPU power.
5. **Efficient Resource Sharing:** Threads share process memory, reducing communication overhead.

5.3 Components of Threads

- **Stack Space:** Stores local variables, function calls, and return addresses specific to the thread.
- **Register Set:** Holds temporary data and intermediate results during thread execution.
- **Program Counter (PC):** Tracks the current instruction being executed by the thread.

5.4 Types of Threads

1. **User-Level Thread (ULT):** Managed entirely by application libraries; kernel is unaware of them. Fast creation and management in user space. *Limitation:* If one thread makes a blocking system call, the entire process blocks.
2. **Kernel-Level Thread (KLT):** Supported directly by the OS kernel. Slower to create and manage, but allows kernel to schedule threads on different processors; if one thread blocks, others continue.

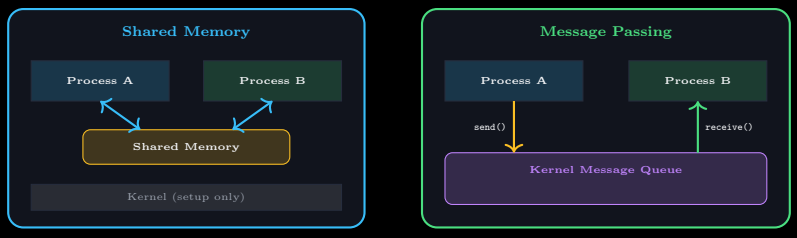
6 Inter-Process Communication (IPC)

IPC Definition: **Inter-Process Communication (IPC)** is a mechanism that allows processes to communicate and share data with each other while they are running.

- Since each process has its own memory space, IPC provides controlled methods for exchanging information.
- Helps processes work together efficiently, safely, and avoid conflicts while accessing shared resources.

6.1 Two Models of IPC

1. **Shared Memory Model:** A region of shared memory is created within the address space of a process. Other processes attach to this shared memory segment. Fastest method (memory speed); synchronization is managed by the application.
2. **Message Passing Model:** Processes exchange messages via the OS kernel without sharing address space. Ideal for distributed and networked environments.



6.2 Message Passing Operations & Links

Primitives: `send(message)` and `receive(message)`. Messages may be of fixed or variable size.

6.2.1 Direct vs. Indirect Communication

- **Direct Communication:** Senders and receivers explicitly name each other:
  - `send(P, message)`: Send a message to Process P.
  - `receive(Q, message)`: Receive a message from Process Q.
- **Indirect Communication:** Messages are sent/received through shared **mailboxes** or **ports**:
  - Each mailbox has a unique ID (e.g., POSIX message queues).
  - `send(A, message)`: Send message to Mailbox A.
  - `receive(A, message)`: Receive message from Mailbox A.

6.2.2 Synchronization Schemes

- **Blocking Send:** Sender is blocked until the message is received by receiver or mailbox.
- **Non-Blocking Send:** Sender sends the message and immediately resumes operation.
- **Blocking Receive:** Receiver blocks until a message is available.
- **Non-Blocking Receive:** Receiver retrieves either a valid message or null.

6.2.3 Buffering Capacity

- **Zero Capacity:** Queue length is 0. Sender must block until receiver receives (rendezvous).
- **Bounded Capacity:** Queue has finite length n. Sender blocks only if queue is completely full.
- **Unbounded Capacity:** Queue length is infinite. Sender never blocks.

7 Process Synchronization & Critical Section Problem

- **Independent Process:** Execution of one process does not affect the execution of others.
- **Cooperative Process:** Execution of one process **affects** the execution of other processes.

Critical Section Problem: A **Critical Section (CS)** is a code segment in a program that accesses shared variables or resources. Only one process can access the critical section at a time; other processes must be in a **wait state**.

7.1 Standard Structure of a Process

```
do {
    /* 1. ENTRY SECTION   (Controls entry into CS, acquires lock on resources) */

        /* 2. CRITICAL SECTION (Accesses shared variables) */

    /* 3. EXIT SECTION    (Releases lock on resources & notifies others) */

    /* 4. REMAINDER SECTION (Non-critical computation) */
} while (true);
```

7.2 Three Mandatory Criteria for Solution to C.S. Problem

1. Mutual Exclusion

If a process is executing in its critical section, no other process is allowed to execute in its critical section.

2. Progress

If no process is executing in its critical section and some processes wish to enter, only those processes that are **not executing in their remainder section** can participate in the decision on which will enter next, and this selection **cannot be postponed indefinitely**.

3. Bounded Waiting

There exists a bound on the number of times other processes are allowed to enter their critical section after a process has made a request to enter, before that request is granted (prevents starvation).

7.3 Two-Process Software Solutions

7.3.1 Algorithm 1: Strict Alternation (Turn Variable)

Runs in user mode. Uses shared integer `int turn;` initialized to 0 or 1.

```
// Code for Process Pi
do {
    while (turn != i);  // Spinlock / Busy waiting (Entry Section)

    /* CRITICAL SECTION */

    turn = j;           // Exit Section

    /* REMAINDER SECTION */
} while (true);
```

- **Mutual Exclusion:** Ensured (since `turn` can only be i or j).
- **Progress: NOT satisfied.** Requires strict alternation. If P_0 exits and does not want to enter again, P_1 cannot enter either because P_0 never sets `turn = 1`.

7.3.2 Algorithm 2: Flag Array (Desire to Enter)

Uses a boolean array `boolean flag[2];` initialized to `false`. `flag[i] = true` indicates P_i is ready to enter CS.

```
// Code for Process Pi
do {
    flag[i] = true;      // Signal intent to enter CS
    while (flag[j]);     // Wait if Pj is also ready

    /* CRITICAL SECTION */

    flag[i] = false;     // Signal exit from CS

    /* REMAINDER SECTION */
} while (true);
```

- **Mutual Exclusion:** Satisfied.
- **Progress & Bounded Waiting: NOT satisfied (Deadlock).** If both P_i and P_j set their flag to `true` simultaneously, both wait forever in the `while` loop.

7.3.3 Algorithm 3: Peterson's Solution

Peterson's solution is a complete two-process software solution. Processes share two variables:

```
int turn;
boolean flag[2];
```

- `turn`: Indicates whose turn it is to enter C.S. (`turn = i` implies turn of Process P_i).
- `flag`: Used to indicate if a process is ready to enter C.S. (`flag[i] = true` implies Process P_i is ready).

Process P_i

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    /* CRITICAL SECTION */

    flag[i] = false;

    /* REMAINDER SECTION */
}
```

Process P_j

```
while (true) {
    flag[j] = true;
    turn = i;
    while (flag[i] && turn == i);

    /* CRITICAL SECTION */

    flag[j] = false;

    /* REMAINDER SECTION */
}
```

Key Properties of Peterson's Solution

- Provides a complete algorithmic solution to C.S. problems for two processes.
- Addresses the requirements of **Mutual Exclusion**, **Progress**, and **Bounded Waiting**.

- **Not guaranteed to work on modern architectures.**
- To improve performance, modern processors and/or compilers may reorder operations that have no dependencies.
- Modern OS systems rely on hardware-supported atomic operations (e.g., `TestAndSet`, `CompareAndSwap`) and memory barriers.